# 1. Features of Java

Java is a high-level, object-oriented programming language developed by Sun Microsystems.

**Important features of Java:**

1. **Simple** – Java has a simple syntax and removes complex features such as pointers and operator overloading.
2. **Object-Oriented** – Java programs are based on classes and objects.
3. **Platform Independent** – Java source code is converted into bytecode, which can run on any system having a JVM.
4. **Portable** – Java bytecode can be transferred and executed on different platforms.
5. **Secure** – Java provides features such as bytecode verification and avoids direct memory access through pointers.
6. **Robust** – Java provides strong memory management, exception handling and garbage collection.
7. **Multithreaded** – Java supports execution of multiple tasks simultaneously using threads.
8. **Distributed** – Java provides networking facilities for developing distributed applications.
9. **Architecture Neutral** – Java bytecode is not dependent on a particular processor architecture.
10. **High Performance** – The Just-In-Time (JIT) compiler improves execution speed.
11. **Dynamic** – Classes can be loaded dynamically when required.

# 2. Bytecode, JDK, JRE and JVM

These four concepts are closely related and should be studied together because several questions in the question bank overlap.

## Bytecode

**Bytecode** is the intermediate code generated by the Java compiler after compiling a Java source program.
For example:

Hello.java
↓
javac Hello.java
↓
Hello.class

The .class file contains Java bytecode.

Bytecode is not directly executed by the computer's hardware. It is executed by the **JVM**.

**JVM – Java Virtual Machine**

The **JVM** is a virtual machine that executes Java bytecode.

**Main functions of JVM:**

- Loads bytecode.
- Verifies bytecode.
- Converts bytecode into machine-level instructions.
- Executes the program.
- Manages memory.
- Performs garbage collection.

**JRE – Java Runtime Environment**

JRE provides the environment required to **run Java programs**.

It mainly contains:

JRE = JVM + Java Runtime Libraries

It does not provide the complete development tools required to create Java programs.

**JDK – Java Development Kit**

JDK is used to **develop, compile and run Java programs**.

It contains:

JDK = JRE + Development Tools

Examples of development tools include:

- javac – Java compiler
- java – Java launcher
- javadoc – documentation generator
- jar – archive tool

Difference between JDK, JRE and JVM

| **JDK** | **JRE** | **JVM** |
|---|---|---|
| Used to develop Java applications | Used to run Java applications | Executes bytecode |
| Contains JRE | Contains JVM | Part of JRE |
| Contains development tools | Contains runtime libraries | Provides execution environment |
| Used by programmers | Used to run applications | Performs bytecode execution |

**Execution of a Java Program**

Java Source Program

↓

javac

↓

Bytecode

(.class)

↓

JVM

↓

Machine Instructions

↓

Output

A Java program is written in a .java file. The Java compiler in the JDK converts the source code into bytecode stored in a .class file. The JRE provides the runtime environment, including the JVM and libraries. The JVM loads and verifies the bytecode and executes it by converting it into machine-level instructions. This process makes Java platform independent.

# 3. Primitive Data Types in Java

Java provides **eight primitive data types**.

| **Data Type** | **Typical Size** | **Example** |
|---|---|---|
| byte | 8 bits | byte age = 20; |
| short | 16 bits | short n = 1000; |
| int | 32 bits | int marks = 75; |
| long | 64 bits | long population = 100000L; |
| float | 32 bits | float avg = 75.5f; |
| double | 64 bits | double price = 125.75; |
| char | 16 bits | char grade = 'A'; |

| Data Type | Typical Size | Example |
|---|---|---|
| boolean | JVM-dependent | boolean pass = true; |

**Categories**

**Integer:** byte, short, int, long
**Floating point:** float, double
**Character:** char
**Boolean:** boolean

# 4. Variables

A **variable** is a named memory location used to store a value that may change during program execution.

**Syntax**

dataType variableName = value;

Example:

int marks = 85;

double average = 78.5;

char grade = 'A';

**Types of variables**

1. **Local variable** – Declared inside a method, constructor or block.
2. **Instance variable** – Declared inside a class but outside methods.
3. **Static variable** – Declared using static and shared by all objects of a class.

# 5. Type Conversion and Type Casting

**Type conversion** means converting a value from one data type to another.
There are two major types.

### A. Widening Conversion

A smaller data type is automatically converted into a larger compatible type.

int a = 10;

double b = a;

Here:

int → double

No explicit casting is required.

**B. Narrowing Conversion / Type Casting**

A larger data type is explicitly converted into a smaller data type.

**Syntax**

dataType variable = (dataType)value;

Example:

```
double d = 25.75;
int n = (int)d;

System.out.println(n);
```

Output:

25

The fractional part .75 is discarded.

**Complete program**

```
class TypeCastingDemo {
   public static void main(String[] args) {

      int num = 25;
      double value = num;       // Widening conversion

      System.out.println("Integer value = " + num);
      System.out.println("Converted double value = " + value);

      double price = 125.75;
      int newPrice = (int) price;   // Narrowing conversion

      System.out.println("Original value = " + price);
      System.out.println("After casting = " + newPrice);
   }
}
```

# 6. Operators in Java

Operators are symbols used to perform operations on variables and values.

**Types of operators**

1. **Arithmetic**

   + - * / %

2. **Relational**
   == != > < >= <=
3. **Logical**
   && || !
4. **Assignment**
   = += -= *= /= %=
5. **Unary**
   ++ --
6. **Conditional/Ternary**
   condition ? value1 : value2
7. **Bitwise**
   & | ^ ~
8. **Shift**
   << >> >>>

# 7. Control Statements and Looping Statements

Control statements determine the order in which statements are executed.

## Types

### 1. Selection statements

Used to make decisions.

if

if-else

else-if

switch

**Example:**

```
if (marks >= 40)
   System.out.println("Pass");
else
```

```
    System.out.println("Fail");
```

**2. Iteration statements**

Used to execute a block repeatedly.

for

while

do-while

**3. Jump statements**

break

continue

return

**Difference between while and do-while**

| **while** | **do-while** |
|---|---|
| Condition checked before execution | Condition checked after execution |
| May execute zero times | Executes at least once |
| Entry-controlled | Exit-controlled |

# 8. Important Application Program – Average of Five Marks

This question combines **variables, data types, operators and type conversion**, so it covers several similar questions in one answer.

```
import java.util.Scanner;


class AverageMarks {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int mark1, mark2, mark3, mark4, mark5;
        int total;
        double average;
```

```
        System.out.print("Enter mark for Subject 1: ");
        mark1 = sc.nextInt();

        System.out.print("Enter mark for Subject 2: ");
        mark2 = sc.nextInt();

        System.out.print("Enter mark for Subject 3: ");
        mark3 = sc.nextInt();

        System.out.print("Enter mark for Subject 4: ");
        mark4 = sc.nextInt();

        System.out.print("Enter mark for Subject 5: ");
        mark5 = sc.nextInt();

        total = mark1 + mark2 + mark3 + mark4 + mark5;

        average = (double) total / 5;

        System.out.println("Total = " + total);
        System.out.println("Average = " + average);

        sc.close();
    }
}
```

**Explanation**

- int is used for storing marks.
- total stores the sum.
- double stores the average.
- (double) total performs explicit type casting.

- / performs division.
- The result is displayed using System.out.println().

# 9. Program Using Control Statements

**Positive, Negative or Zero**

```
import java.util.Scanner;

class NumberCheck {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int num = sc.nextInt();

      if (num > 0)
         System.out.println("Positive number");
      else if (num < 0)
         System.out.println("Negative number");
      else
         System.out.println("Zero");

      sc.close();
   }
}
```

# 10. Multiplication Table Using Loop

```
import java.util.Scanner;


class MultiplicationTable {

   public static void main(String[] args) {


      Scanner sc = new Scanner(System.in);


      System.out.print("Enter a number: ");

      int n = sc.nextInt();
```

```
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }

        sc.close();
    }
}
```

# 11. Prime Number Program

```
import java.util.Scanner;

class PrimeNumber {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        boolean prime = true;

        if (n <= 1) {
            prime = false;
        } else {
            for (int i = 2; i <= n / 2; i++) {
                if (n % i == 0) {
                    prime = false;
                    break;
                }
            }
        }

        if (prime)
            System.out.println(n + " is a prime number");
        else
            System.out.println(n + " is not a prime number");

        sc.close();
    }
}
```

**ARRAYS AND STRINGS**

# 12. Arrays

An **array** is a collection of elements of the same data type stored under a single variable name.

**Declaration**

int marks[];

**Creation**

marks = new int[5];

Or:

int marks[] = {80, 75, 90, 85, 70};

**Important features**

- Stores multiple values.
- All elements have the same data type.
- Index starts from 0.
- Array size is fixed after creation.
- length gives the number of elements.

**Program – Total, Average and Highest Mark**

```
import java.util.Scanner;


class StudentMarks {
   public static void main(String[] args) {


      Scanner sc = new Scanner(System.in);


      int[] marks = new int[5];
      int total = 0;
      int highest;


      for (int i = 0; i < marks.length; i++) {
         System.out.print("Enter mark " + (i + 1) + ": ");
         marks[i] = sc.nextInt();
```

```
            total = total + marks[i];
        }

        highest = marks[0];

        for (int i = 1; i < marks.length; i++) {
            if (marks[i] > highest)
                highest = marks[i];
        }

        double average = (double) total / marks.length;

        System.out.println("Total = " + total);
        System.out.println("Average = " + average);
        System.out.println("Highest mark = " + highest);

        sc.close();
    }
}
```

# 13. Strings

A **String** is a sequence of characters. In Java, String is an object of the String class.

Example:

String name = "Java";

**Common String methods**

| **Method** | **Purpose** |
|---|---|
| **length()** | **Returns number of characters** |
| **charAt()** | **Returns character at specified position** |
| **equals()** | **Compares strings** |
| **equalsIgnoreCase()** | **Compares ignoring case** |

| Method | Purpose |
|---|---|
| **substring()** | **Extracts part of a string** |
| **toUpperCase()** | **Converts to uppercase** |
| **toLowerCase()** | **Converts to lowercase** |
| **concat()** | **Joins strings** |

**Program – String Operations**

```
import java.util.Scanner;

class StringOperations {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first string: ");
        String str1 = sc.nextLine();

        System.out.print("Enter second string: ");
        String str2 = sc.nextLine();

        System.out.println("Length = " + str1.length());

        String reverse = "";

        for (int i = str1.length() - 1; i >= 0; i--) {
            reverse = reverse + str1.charAt(i);
        }

        System.out.println("Reverse = " + reverse);
```

```
        if (str1.equals(str2))
            System.out.println("Both strings are equal");
        else
            System.out.println("Strings are different");

        sc.close();
    }
}
```

# 14. Count Vowels and Consonants

```
import java.util.Scanner;

class VowelConsonant {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        int vowels = 0;
        int consonants = 0;

        for (int i = 0; i < str.length(); i++) {

            char ch = Character.toLowerCase(str.charAt(i));

            if (ch >= 'a' && ch <= 'z') {

                if (ch == 'a' || ch == 'e' ||
                    ch == 'i' || ch == 'o' || ch == 'u') {
```

```
                vowels++;
            } else {
                consonants++;
            }
        }
    }

    System.out.println("Vowels = " + vowels);
    System.out.println("Consonants = " + consonants);

    sc.close();
  }
}
```

# 15. Class and Object

A **class** is a blueprint or template used to create objects.

An **object** is an instance of a class.

Example:

```
class Student {
  int rollNo;
  String name;

  void display() {
    System.out.println(rollNo);
    System.out.println(name);
  }
}
```

Objects are created using new.

```
Student s1 = new Student();
```

# 16. Access Specifiers

Access specifiers control the accessibility of class members.

**Four access levels**

| Access Specifier | Same Class | Same Package | Subclass | Other Package |
|---|---|---|---|---|
| private | Yes | No | No | No |
| default | Yes | Yes | No* | No |
| protected | Yes | Yes | Yes | Yes, through inheritance |
| public | Yes | Yes | Yes | Yes |

*Default/package-private access is available within the same package.

Example

```
class Student {

    private int rollNo;
    protected String name;
    public String course;

    void setRollNo(int r) {
        rollNo = r;
    }

    void display() {
        System.out.println("Roll No = " + rollNo);
        System.out.println("Name = " + name);
        System.out.println("Course = " + course);
    }
}

class AccessDemo {
    public static void main(String[] args) {
```

```
        Student s = new Student();

        s.setRollNo(101);
        s.name = "Anu";
        s.course = "MCA";

        s.display();
    }
}
```

**Important:** A private member cannot be directly accessed outside its class. It is normally accessed through methods such as setters and getters.

# 17. this Keyword

The this keyword refers to the **current object**.

One of its most common uses is to distinguish an instance variable from a parameter having the same name.

```
class Student {

    int rollNo;
    String name;

    Student(int rollNo, String name) {

        this.rollNo = rollNo;
        this.name = name;
    }

    void display() {
        System.out.println("Roll No = " + rollNo);
        System.out.println("Name = " + name);
    }
```

```
}


class ThisDemo {
   public static void main(String[] args) {


      Student s = new Student(101, "Anu");


      s.display();
   }
}
```

**Explanation**

In:

this.rollNo = rollNo;

- this.rollNo refers to the instance variable.
- rollNo refers to the constructor parameter.

Other uses of this include:

- Calling current class methods.
- Calling another constructor using this().
- Passing the current object as an argument.
- Returning the current object.

# 18. Static Variable and Static Method

A **static variable** belongs to the class rather than individual objects. Only one copy is maintained and shared by all objects.

A **static method** belongs to the class and can be called using the class name.

## Example

```
class BankAccount {


   static String bankName = "ABC Bank";
```

```
    int accountNo;
    String name;

    BankAccount(int accountNo, String name) {
        this.accountNo = accountNo;
        this.name = name;
    }

    static void displayBankName() {
        System.out.println("Bank Name = " + bankName);
    }

    void display() {
        System.out.println("Account No = " + accountNo);
        System.out.println("Name = " + name);
    }
}

class StaticDemo {
    public static void main(String[] args) {

        BankAccount a1 = new BankAccount(101, "Anu");
        BankAccount a2 = new BankAccount(102, "Ravi");

        BankAccount.displayBankName();

        a1.display();
        a2.display();
    }
}
```

**Key point**

static variable → common to all objects

instance variable → separate copy for each object

# 19. Nested and Inner Classes

A class declared inside another class is called a **nested class**.

Example:

```
class Calculator {

    class Operations {

        void add(int a, int b) {
            System.out.println("Sum = " + (a + b));
        }
    }
}

class NestedDemo {
    public static void main(String[] args) {

        Calculator c = new Calculator();

        Calculator.Operations op = c.new Operations();

        op.add(10, 20);
    }
}
```

**Important distinction**

A **non-static nested class** is commonly called an **inner class**.

For a non-static inner class:

Outer obj = new Outer();

Outer.Inner inner = obj.new Inner();

A static nested class can be created without creating an object of the outer class:

Outer.Inner obj = new Outer.Inner();

# 20. Method Overloading

**Method overloading** means defining multiple methods with the same name but different parameter lists in the same class.

The difference may be in:

- Number of parameters.
- Type of parameters.
- Order of parameters.

**Example**

```
class Area {


   double calculate(double radius) {
      return 3.14 * radius * radius;
   }


   double calculate(double length, double breadth) {
      return length * breadth;
   }


   double calculate(double base, double height, int triangle) {
      return 0.5 * base * height;
   }
}


class OverloadDemo {
   public static void main(String[] args) {


      Area a = new Area();
```

```
        System.out.println("Circle = " + a.calculate(5));
        System.out.println("Rectangle = " + a.calculate(10, 5));
        System.out.println("Triangle = " + a.calculate(10, 5, 1));
    }
}
```

### How does Java identify the method?

The compiler examines the **number, type and order of arguments** and selects the matching method.

Method overloading is an example of **compile-time polymorphism**.

# 21. Constructor and Constructor Overloading

A **constructor** is a special member of a class used to initialize objects.

**Characteristics**

- Constructor has the same name as the class.
- It has no return type.
- It is automatically called when an object is created.
- Constructors can be overloaded.
- Constructors are not inherited.

**Constructor overloading**

Having multiple constructors with different parameter lists is called **constructor overloading**.

```
class Student {

    int rollNo;
    String name;

    Student() {
        rollNo = 0;
        name = "Unknown";
    }
```

```
    Student(int rollNo) {
        this.rollNo = rollNo;
        name = "Unknown";
    }

    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    void display() {
        System.out.println(rollNo + " " + name);
    }
}

class ConstructorDemo {
    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student(101);
        Student s3 = new Student(102, "Anu");

        s1.display();
        s2.display();
        s3.display();
    }
}
```

# 22. Inheritance

Inheritance is the mechanism by which one class acquires the properties and methods of another class.

**Syntax**

```
class Child extends Parent {

}
```

**Types**

Java directly supports:

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance

Java does **not** support multiple inheritance through classes because a class cannot extend multiple classes. Multiple inheritance of behaviour can be achieved using interfaces.

**Example**

```
class Employee {


   void display() {
      System.out.println("Employee details");
   }
}


class Manager extends Employee {


   void display() {
      System.out.println("Manager details");
   }
}


class InheritanceDemo {
   public static void main(String[] args) {


      Manager m = new Manager();
      m.display();
   }
}
```

# 23. Method Overriding

Method overriding occurs when a subclass provides its own implementation of a method already defined in the superclass.

**Rules**

- Same method name.
- Same parameter list.
- Compatible return type.
- Requires inheritance.
- The overriding method cannot have more restrictive access.
- final methods cannot be overridden.

**Example**

```
class Vehicle {


   void display() {
      System.out.println("Vehicle");
   }
}


class Car extends Vehicle {


   @Override
   void display() {
      System.out.println("Car");
   }
}


class Bike extends Vehicle {


   @Override
   void display() {
      System.out.println("Bike");
```

```
    }
}
```

# 24. Dynamic Method Dispatch

**Dynamic Method Dispatch** is the mechanism by which a superclass reference is used to refer to a subclass object and the overridden method is selected at **runtime**.

**Example**

```
class Vehicle {

    void display() {
        System.out.println("Vehicle");
    }
}

class Car extends Vehicle {

    @Override
    void display() {
        System.out.println("Car");
    }
}

class Bike extends Vehicle {

    @Override
    void display() {
        System.out.println("Bike");
    }
}

class DynamicDemo {
    public static void main(String[] args) {
```

```
        Vehicle v;

        v = new Car();
        v.display();

        v = new Bike();
        v.display();
    }
}
```

### Output

```
Car
Bike
```

Although v is a Vehicle reference, the method executed depends on the actual object.

Vehicle reference

↓

Car object → Car.display()

Vehicle reference

↓

Bike object → Bike.display()

# 25. Abstract Class

An **abstract class** is a class declared using the abstract keyword. It may contain abstract methods as well as concrete methods.

An abstract method has a declaration but no implementation.

```
abstract class Shape {

    abstract void area();
```

```
    void display() {
        System.out.println("This is a shape");
    }
}
```

An abstract class cannot normally be instantiated directly.

**Example**

```
abstract class Shape {

    abstract void area();
}

class Circle extends Shape {

    double radius = 5;

    void area() {
        System.out.println("Area = " + (3.14 * radius * radius));
    }
}

class Rectangle extends Shape {

    double length = 10;
    double breadth = 5;

    void area() {
        System.out.println("Area = " + (length * breadth));
    }
}
```

```
class AbstractDemo {
   public static void main(String[] args) {

      Shape s;

      s = new Circle();
      s.area();

      s = new Rectangle();
      s.area();
   }
}
```

# 26. Interface

An **interface** is a reference type used to define a contract that implementing classes must follow.

**Syntax**

```
interface Payment {

   void pay();
}
```

A class implements an interface using:

```
class CreditCard implements Payment
```

Example

```
interface Payment {

   void pay();
}

class CreditCard implements Payment {

   public void pay() {
```

```
        System.out.println("Payment using Credit Card");
    }
}

class UPI implements Payment {

    public void pay() {
        System.out.println("Payment using UPI");
    }
}

class InterfaceDemo {
    public static void main(String[] args) {

        Payment p;

        p = new CreditCard();
        p.pay();

        p = new UPI();
        p.pay();
    }
}
```

**Abstract class vs Interface**

| **Abstract Class** | **Interface** |
|---|---|
| Declared using abstract class | Declared using interface |
| Extended using extends | Implemented using implements |
| Can have constructors | Cannot have constructors |
| Can have instance variables | Fields are constants by default |

| Abstract Class | Interface |
| --- | --- |
| A class can extend only one class | A class can implement multiple interfaces |

# 27. Package

A **package** is a group of related classes and interfaces.

**Advantages**

- Organizes classes.
- Avoids naming conflicts.
- Provides access protection.
- Makes programs easier to maintain.

**Declaring a package**

```
package studentinfo;


public class Student {


   public void display() {
      System.out.println("Student Information");
   }
}
```

Using the package

```
import studentinfo.Student;


class TestStudent {


   public static void main(String[] args) {


      Student s = new Student();
      s.display();
   }
}
```

**Important point**

The package statement should normally appear at the beginning of the source file.

# 28. Sub-package

A **sub-package** is a package organized hierarchically under another package.

For example:

college

```
├── student
└── staff
```

Here college.student and college.staff are separate packages.

**Student package**

```
package college.student;


public class Student {
   public void display() {
      System.out.println("Student details");
   }
}
```

**Main program**

```
import college.student.Student;


class Main {
   public static void main(String[] args) {


      Student s = new Student();
      s.display();
   }
}
```

**Important:** A sub-package is treated as a separate package. Members of the parent package are not automatically available to a sub-package.

# 29. Exception

An **exception** is an abnormal condition that occurs during program execution and interrupts the normal flow of a program.

Examples:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- IOException
- NumberFormatException

**Two broad categories**

**Checked exceptions:** Checked by the compiler.

Example:

IOException

**Unchecked exceptions:** Occur during runtime.

Example:

ArithmeticException

NullPointerException

# 30. try-catch

The try-catch statement is used to handle exceptions so that abnormal conditions do not unnecessarily terminate the program.

**Example**

**29. Exception**

An **exception** is an abnormal condition that occurs during program execution and interrupts the normal flow of a program.

Examples:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- IOException
- NumberFormatException

**Two broad categories**

**Checked exceptions:** Checked by the compiler.

Example:

IOException

**Unchecked exceptions:** Occur during runtime.

Example:

ArithmeticException

NullPointerException

The try-catch statement is used to handle exceptions so that abnormal conditions do not unnecessarily terminate the program.

**Example**

```
class ExceptionDemo {


   public static void main(String[] args) {


      try {


         int a = 10;

         int b = 0;


         int result = a / b;


         System.out.println(result);


      } catch (ArithmeticException e) {


         System.out.println("Cannot divide by zero");

      }

   }

}
```

**Working**

try block

↓

Exception occurs

↓

Matching catch block

↓

Exception handled

# 31. throw and throws

These two keywords are frequently confused.

**throw**

throw is used to **explicitly generate an exception**.

Example:

```
throw new ArithmeticException("Invalid operation");
```

**throws**

throws is used in a method declaration to indicate that a method may pass an exception to its caller.

Example:

```
void readFile() throws IOException {
   // code
}
```

**Difference**

| **throw** | **throws** |
|---|---|
| Used to explicitly throw an exception | Declares possible exceptions |
| Used inside method/block | Used in method declaration |
| Throws one exception object at a time | Can declare multiple exceptions |
| Followed by an exception object | Followed by exception class names |

# 32. User-Defined Exception Using throw

```
class AgeException extends Exception {


   AgeException(String message) {
      super(message);
```

```
    }
}

class ValidateAge {

    static void checkAge(int age) throws AgeException {

        if (age < 18) {
            throw new AgeException("Age must be 18 or above");
        }

        System.out.println("Eligible");
    }

    public static void main(String[] args) {

        try {
            checkAge(16);
        }
        catch (AgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

**Explanation**

1. AgeException is a user-defined exception.
2. checkAge() checks the condition.
3. throw creates and throws the exception.
4. throws declares that the method may generate AgeException.
5. catch handles the exception.

# 33. finally Block

The finally block contains statements that are normally executed whether an exception occurs or not.

It is commonly used for cleanup operations such as closing resources.

**Example**

```
class FinallyDemo {


   public static void main(String[] args) {


      try {


         int result = 10 / 2;

         System.out.println("Result = " + result);


      }

      catch (ArithmeticException e) {


         System.out.println("Exception occurred");

      }

      finally {


         System.out.println("Finally block executed");

      }

   }

}
```

**tructure**

```
try {

   // risky code

}

catch (Exception e) {

   // exception handling
```

```
}

finally {

   // cleanup code

}
```

# 34. Combined Exception Handling Program

This program covers the overlapping questions involving **try-catch, throw, throws and finally**.

```
class BankException extends Exception {

   BankException(String message) {
      super(message);
   }
}

class BankAccount {

   double balance = 5000;

   void withdraw(double amount) throws BankException {

      if (amount <= 0) {
         throw new BankException("Invalid withdrawal amount");
      }

      if (amount > balance) {
         throw new BankException("Insufficient balance");
      }

      balance = balance - amount;

      System.out.println("Withdrawal successful");
      System.out.println("Remaining balance = " + balance);
   }
}

class BankDemo {

   public static void main(String[] args) {

      BankAccount account = new BankAccount();

      try {

         account.withdraw(6000);

      }
      catch (BankException e) {

         System.out.println("Exception: " + e.getMessage());
```

```
        }
        finally {

            System.out.println("Transaction completed.");
        }
    }
}
```

### Concepts demonstrated

- Class and object
- Instance variable
- Method
- User-defined exception
- throw
- throws
- try
- catch
- finally
- Exception handling

# 35. Bank Account System Using Inheritance, Interface and Exception Handling

```
interface Transaction {


    void deposit(double amount);


    void withdraw(double amount) throws Exception;
}


class BankAccount implements Transaction {


    protected String name;

    protected double balance;


    BankAccount(String name, double balance) {

        this.name = name;

        this.balance = balance;
```

```
    }

    public void deposit(double amount) {

        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Deposited = " + amount);
        }
    }

    public void withdraw(double amount) throws Exception {

        if (amount <= 0) {
            throw new Exception("Invalid withdrawal amount");
        }

        if (amount > balance) {
            throw new Exception("Insufficient balance");
        }

        balance = balance - amount;

        System.out.println("Withdrawn = " + amount);
    }

    void display() {
        System.out.println("Name = " + name);
        System.out.println("Balance = " + balance);
    }
}
```

```
class SavingsAccount extends BankAccount {

    SavingsAccount(String name, double balance) {
        super(name, balance);
    }

    @Override
    void display() {
        System.out.println("Savings Account");
        super.display();
    }
}

class BankApplication {

    public static void main(String[] args) {

        SavingsAccount account =
            new SavingsAccount("Anu", 10000);

        try {

            account.deposit(2000);
            account.withdraw(5000);

        }
        catch (Exception e) {

            System.out.println("Error: " + e.getMessage());

        }
```

```
        finally {

            System.out.println("Final Account Details");
            account.display();
        }
    }
}
```